

Restore span's initializer_list constructor for C++29

Document #: P4190R0
Date: 2026-04-16
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mark.hoemmen-MILD-OBfuscation@gmail.com>
Hana Dusíková
<hanicka-MILD-OBfuscation@hanicka.net>
Rob Parolin
<robertoparolin-MILD-OBfuscation@gmail.com>

Contents

- 1 Revision history
- 2 Abstract
- 3 Why the constructor was removed
- 4 Adding the constructor back, but with constraints
- 5 This is still a breaking change
- 6 Should we adjust the constraints to permit “reasonable” conversions?
- 7 Implementation
- 8 Proposed wording
 - 8.1 Add `__cpp_lib_span_initializer_list` macro
 - 8.2 Add `initializer_list` include to `[span.syn]`
 - 8.3 Add `initializer_list` constructor to `span`'s class declaration
 - 8.4 Add `initializer_list` constructor to `span`'s description
 - 8.5 Add a new section `[diff.cpp26.containers]` section
- 9 Appendix A: Implementation listing

§ 1 Revision history

- R0 to be submitted for the 2026-04 post-Croydon mailing

§ 2 Abstract

P4144R1 reverted P2447R6 by removing span's `initializer_list` constructor. We propose to add it back in C++29, but with constraints this time, in order to avoid the unfortunate conversions that led to its removal.

§ 3 Why the constructor was removed

P2447R6 proposed adding the following span constructor for C++26.

```
constexpr explicit(extent != dynamic_extent)
span(std::initializer_list<value_type> il);
```

As P4144R1 explains, the new constructor's lack of constraints led to silent changes in behavior from C++23 to C++26. P2447 gave some examples, but the following example from P4144 motivated WG21 enough to reconsider the feature.

```
#include <span>

int main() {
    bool data[4] = {true, false, true, false};
    bool* ptr = data;
    size_t size = 4;

    std::span<bool> span_nc{ptr, size};
    // OK in both C++23 and C++26
    assert(span_nc.size() == size);

    std::span<const bool> bad_span{ptr, size};
    // OK in C++23, but FAILS in C++26
    assert(bad_span.size() == size);
    return 0;
}
```

WG21 thus voted at the Croydon meeting in March 2026 to adopt P4144R1, which removed the new constructor entirely.

§ 4 Adding the constructor back, but with constraints

Coauthor Hana Dusíková came up with a way to constrain the constructor generically (for all T) so that it refuses to convert a pointer to `value_type`.

```
template <same_as<value_type> InitListValueType>
constexpr
    my_span(std::initializer_list<InitListValueType> il)
        requires (is_const_v<ElementType>);
```

This would prohibit the troublesome cases discussed in P4144R1. However, it would also change the original P2447R6 design by forcing the type match to be exact. For example, `span<const bool>{true, false, true}` would work, but `span<const bool>{1, 0, 1}` would fail overload resolution.

§ 5 This is still a breaking change

Adding the constructor is still a breaking change, as the following two examples from P2447R6 show. The difference is that this proposal would ensure that the common case of a pointer and a size would always resolve to the `span(It first, size_type count)` constructor.

```
void one(pair<int, int>); // 1
void one(span<const int>); // 2
void t1() { one({1, 2}); } // ambiguous between 1 and 2
```

In C++26, `one({1, 2})` would call the `pair<int, int>` overload. After adoption of this proposal, `one({1, 2})` would be ambiguous. This is no different than what adoption of P2447R6 for C++26 would have caused for valid C++23 code.

```
void two(span<const int, 2>);
void t2() { two({{1, 2}}); } // ill-formed; previously well-formed
```

In C++26, `two({{1, 2}})` would be well-formed. After adoption of this proposal, `two({{1, 2}})` would become ill-formed. This is no different than what adoption of P2447R6 for C++26 would have caused for valid C++23 code.

We do not think the proposed change would affect overload resolution for any of the other constructors. For example, the `span(It first, End last)` constructor constrains `End` not to be convertible to `size_t`, so `span(pointer, size)` would never resolve to this anyway.

§ 6 Should we adjust the constraints to permit “reasonable” conversions?

The constructor’s `same_as<value_type>` constraint excludes some use cases that would have an unambiguous meaning. For example:

- `span<const bool>{true, false, true}` would work, but `span<const bool>{1, 0, 1}` would fail overload resolution; and
- `span<const float>{1.0f, 2.0f, 3.0f}` would work, but `span<const float>{1, 2, 3}` and `span<const float>{1.0, 2.0, 3.0}` would fail overload resolution.

One could imagine relaxing the constraint to permit conversions between arithmetic types. More generally, one could imagine trying to exclude all cases that might change overload resolution from any of the existing constructors to the `initializer_list` constructor.

We do not propose relaxing the constraint, because doing so would overly complicate span. A span is a pointer and a size. The more constructors we add to it, the less clear this becomes.

§ 7 Implementation

NVIDIA's libcu++ library is a Standard Library implementation. [Pull Request 8314](#) implements the proposed change in libc++.

[This Compiler Explorer link](#) shows a full span implementation with some tests. Appendix A shows the implementation.

§ 8 Proposed wording

§ 8.1 Add `__cpp_lib_span_initializer_list` macro

[Editor's note: In [version.syn], add the `__cpp_lib_span_initializer_list` feature test macro definition. Adjust the placeholder value as needed so as to denote this proposal's date of adoption.]

```
#define __cpp_lib_span 202311L //
    freestanding, also in <span>

#define __cpp_lib_span_initializer_list YYMMML //
    freestanding, also in <span>

#define __cpp_lib_spanstream 202106L // also in
    <iosfwd>, <spanstream>
```

§ 8.2 Add `initializer_list` include to [span.syn]

[Editor's note: Add `#include <initializer_list>` to [span.syn] (“Header synopsis”) as shown below.]

```
#include <initializer_list> // see [initializer.list.syn]

// mostly freestanding
namespace std {
```

§ 8.3 Add `initializer_list` constructor to span's class declaration

[Editor's note: Add `span(initializer_list<value_type>)` constructor to span's class declaration in [span.overview], as shown below.]

```
// [span.cons], constructors, copy, and assignment
constexpr span() noexcept;
template<class It>
```

```

    constexpr explicit(extent != dynamic_extent) span(It first,
        size_type count);
template<class It, class End>
    constexpr explicit(extent != dynamic_extent) span(It first, End
        last);
template<size_t N>
    constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<class T, size_t N>
    constexpr span(array<T, N>& arr) noexcept;
template<class T, size_t N>
    constexpr span(const array<T, N>& arr) noexcept;
template<class R>
    constexpr explicit(extent != dynamic_extent) span(R&& r);

```

```

constexpr explicit(extent != dynamic_extent)
    span(std::initializer_list<value_type> il);

```

```

constexpr span(const span& other) noexcept = default;
template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below)
        span(const span<OtherElementType, OtherExtent>& s) noexcept;

```

§ 8.4 Add `initializer_list` constructor to `span`'s description

[Editor's note: Add `span(initializer_list<value_type>)` constructor description to `[span.cons]`.]

19 *Effects:* Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

20 *Throws:* What and when `ranges::data(r)` and `ranges::size(r)` throw.

```

template<class InitListValueType>
    constexpr explicit(extent != dynamic_extent)
        span(std::initializer_list<InitListValueType> il);

```

21 *Constraints:*

(21.1) — `same_as<value_type, InitListValueType>` is true.

(21.2) — `is_const_v<element_type>` is true.

22 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `il.size() == extent` is true.

23 *Effects:* Initializes `data_` with `il.data()` and `size_` with `il.size()`.

```

constexpr span(const span& other) noexcept = default;

```

24 *Postconditions:* `other.size() == size()` && `other.data() == data()`.

§ 8.5 Add a new section `[diff.cpp26.containers]` section

[Editor's note: Add a new [diff.cpp26.containers] section, as shown below.]

1 Affected subclause: [span.overview]

Change: `span<const T>` is constructible from `initializer_list<T>`.

Rationale: Permit passing a braced initializer list to a function taking `span`.

Effect on original feature: Valid C++ 2026 code that relies on the lack of this constructor may refuse to compile, or change behavior in this revision of C++.

[Example 1:

```
void one(pair<int, int>);           // #1
void one(span<const int>);         // #2
void t1() { one({1, 2}); }         // ambiguous between #1 and #2; previously
    called #1

void two(span<const int, 2>);
void t2() { two({{1, 2}}); }      // ill-formed; previously well-formed
```

— end example]

§ 9 Appendix A: Implementation listing

```
#include <any>
#include <cassert>
#include <stdexcept>
#include <iterator>
#include <span>

#define PROPOSED_CHANGE 1

namespace std2 {

    using std::dynamic_extent;
    using std::ptrdiff_t;
    using std::size_t;

    template<class ElementType, size_t Extent = dynamic_extent>
    class span;

    namespace impl {
        template<class T>
        constexpr bool is_span_v = false;
        template<class ElementType, size_t Extent>
        constexpr bool is_span_v<span<ElementType, Extent>> = true;
        template<class ElementType, size_t Extent>
        constexpr bool is_span_v<
            std::span<ElementType, Extent>> = true;

        template<class T>
        constexpr bool is_std_array_v = false;
```

```

template<class ValueType, size_t Extent>
constexpr bool is_std_array_v<
    std::array<ValueType, Extent>> = true;
} // namespace impl

template<class ElementType, size_t Extent>
class span {
private:
    ElementType* data_ = nullptr;
    size_t size_ = Extent == dynamic_extent ? size_t() : Extent;

public:
    // constants and types
    using element_type = ElementType;
    using value_type = std::remove_cv_t<ElementType>;
    using size_type = size_t;
    using difference_type = ptrdiff_t;
    using pointer = element_type*;
    using const_pointer = const element_type*;
    using reference = element_type&;
    using const_reference = const element_type&;
    using iterator = pointer;
    using const_iterator = const_pointer;
    using reverse_iterator = std::reverse_iterator<iterator>;
    // libcpp doesn't seem to define std::const_iterator or
    // std::make_const_iterator in <iterator>. We don't actually
    // need these, though.
    using const_reverse_iterator =
        std::reverse_iterator<const_iterator>;
    static constexpr size_type extent = Extent;

    // [span.cons], constructors, copy, and assignment
    constexpr span() noexcept
        requires(Extent == dynamic_extent || Extent == 0) = default;

    template<class It>
    requires(
        std::contiguous_iterator<It> &&
        std::is_convertible_v<
            std::remove_reference_t<std::iter_reference_t<It>>(*)[],
            element_type(*)[]
        >
    )
    constexpr explicit(extent != dynamic_extent)
        span(It first, size_type count)
        : data_(::std::to_address(first)), size_(count) {}

    template<class It, class End>
    requires(
        std::is_convertible_v<
            std::remove_reference_t<std::iter_reference_t<It>>(*)[],
            element_type(*)[]
        > &&
        std::contiguous_iterator<
            std::remove_reference_t<std::iter_reference_t<It>>

```

```

> &&
std::size_sentinel_for<End, It> &&
! std::is_convertible_v<End, size_t>
)
constexpr explicit(extent != dynamic_extent)
span(It first, End last)
    : data_(&std::to_address(first)),
      size_(last - first)
{}

template<size_t N>
constexpr span(
    std::type_identity_t<element_type> (&arr)[N]) noexcept
requires(
    (extent == dynamic_extent || N == extent) &&
    std::is_convertible_v<
        std::remove_pointer_t<decltype(std::data(arr))>(*)[],
        element_type(*)[]
    >
) : data_(arr), size_(N) {}

template<class T, size_t N>
constexpr span(std::array<T, N>& arr) noexcept
requires(
    (extent == dynamic_extent || N == extent) &&
    std::is_convertible_v<
        std::remove_pointer_t<decltype(std::data(arr))>(*)[],
        element_type(*)[]
    >
) : data_(arr.data()), size_(N) {}

template<class T, size_t N>
constexpr span(const std::array<T, N>& arr) noexcept
requires(
    (extent == dynamic_extent || N == extent) &&
    std::is_convertible_v<
        std::remove_pointer_t<decltype(std::data(arr))>(*)[],
        element_type(*)[]
    >
) : data_(arr.data()), size_(N) {}

template<class R> requires(
    std::ranges::contiguous_range<
        std::remove_reference_t<
            std::ranges::range_reference_t<R>
        >
    > &&
    std::ranges::sized_range<
        std::remove_reference_t<
            std::ranges::range_reference_t<R>
        >
    > &&
    (
        std::ranges::borrowed_range<
            std::remove_reference_t<

```



```

        std::ranges::range_reference_t<R>
    >
    > ||
    std::is_const_v<element_type>
) &&
! impl::is_span_v<std::remove_cvref_t<R>> &&
! impl::is_std_array_v<std::remove_cvref_t<R>> &&
! std::is_array_v<std::remove_cvref_t<R>> &&
std::is_convertible_v<
    std::remove_reference_t<
        std::ranges::range_reference_t<R>
    >(*)[],
    element_type(*)[]
>
)
constexpr explicit(extent != dynamic_extent)
span(R&& r)
    : data_( ::std::ranges::data(r)),
      size_( ::std::ranges::size(r))
{}

#if defined(PROPOSED_CHANGE)
template<std::same_as<value_type> InitListValueType>
constexpr explicit(extent != dynamic_extent)
span( ::std::initializer_list<InitListValueType> il)
    requires(std::is_const_v<element_type>)
    : data_(std::data(il)), size_(il.size())
{
    assert(size_ == 3);
}
#endif

constexpr span(const span& other) noexcept = default;

template<class OtherElementType, size_t OtherExtent>
requires(
    extent == dynamic_extent ||
    OtherExtent == dynamic_extent ||
    extent == OtherExtent
)
constexpr
explicit(
    extent != dynamic_extent &&
    OtherExtent == dynamic_extent
)
span(const span<OtherElementType, OtherExtent>& s) noexcept
    : data_(s.data()), size_(s.size())
{}

constexpr span& operator=(const span& other) noexcept = default;

// [span.sub], subviews
template<size_t Count>
constexpr span<element_type, Count> first() const {
    return span<element_type, Count>(data(), Count);
}

```

```

    }
    template<size_t Count>
    constexpr span<element_type, Count> last() const {
        return span<element_type, Count>(
            data() + (size() - Count),
            Count
        );
    }
}

template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type,
    Count != dynamic_extent ? Count
    : (Extent != dynamic_extent ? Extent - Offset
    : dynamic_extent)
> subspan() const {
    static_assert(
        Offset <= Extent &&
        (Count == dynamic_extent || Count <= Extent - Offset)
    );
    static constexpr size_t ResultExtent =
        Count != dynamic_extent ? Count
        : (Extent != dynamic_extent ? Extent - Offset
        : dynamic_extent);
    return span<ElementType, ResultExtent>(
        data() + Offset,
        Count != dynamic_extent ? Count : size() - Offset
    );
}

constexpr span<element_type, dynamic_extent>
first(size_type count) const {
    return span<element_type, dynamic_extent>(data(), count);
}

constexpr span<element_type, dynamic_extent>
last(size_type count) const {
    return span<element_type, dynamic_extent>(
        data() + (size() - count),
        count
    );
}

constexpr span<element_type, dynamic_extent> subspan(
    size_type offset, size_type count = dynamic_extent) const {
    return span<element_type, dynamic_extent>(
        data() + offset,
        count == dynamic_extent ? size() - offset : count
    );
}

// [span.obs], observers
constexpr size_type size() const noexcept {
    return Extent == dynamic_extent ? size_ : Extent;
}

constexpr size_type size_bytes() const noexcept {
    return size() * sizeof(element_type);
}

constexpr bool empty() const noexcept {

```

```

    return size() == 0;
}

// [span.elem], element access
constexpr reference operator[] (size_type idx) const {
    return data_[idx];
}
constexpr reference at(size_type idx) const {
    if (idx >= size()) {
        throw std::out_of_range("span::at: Index out of bounds");
    }
    return data_[idx];
}
constexpr reference front() const {
    return *data();
}
constexpr reference back() const {
    return *(data() + (size() - 1));
}
constexpr pointer data() const noexcept {
    return data_;
}

// [span.iterators], iterator support
constexpr iterator begin() const noexcept {
    return data();
}
constexpr iterator end() const noexcept {
    return data() + size();
}
constexpr const_iterator cbegin() const noexcept {
    return begin();
}
constexpr const_iterator cend() const noexcept {
    return end();
}
constexpr reverse_iterator rbegin() const noexcept {
    return reverse_iterator(end());
}
constexpr reverse_iterator rend() const noexcept {
    return reverse_iterator(begin());
}
constexpr const_reverse_iterator
    crbegin() const noexcept {
    return rbegin();
}
constexpr const_reverse_iterator
    crend() const noexcept {
    return rend();
}
};

// [span.objectrep], views of object representation
template<class ElementType, size_t Extent>
requires(

```

```

        ! ::std::is_volatile_v<ElementType>
    )
    span<
        const std::byte,
        Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) *
        Extent
    >
    as_bytes(span<ElementType, Extent> s) noexcept {
        using R = span<
            const std::byte,
            Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) *
            Extent
        >;
        return R(
            reinterpret_cast<const std::byte*>(s.data()),
            s.size_bytes()
        );
    }

    template<class ElementType, size_t Extent>
    requires(
        ! ::std::is_const_v<ElementType> &&
        ! ::std::is_volatile_v<ElementType>
    )
    span<
        std::byte,
        Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) *
        Extent
    >
    as_writable_bytes(span<ElementType, Extent> s) noexcept {
        using R = span<
            std::byte,
            Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) *
            Extent
        >;
        return R(
            reinterpret_cast<std::byte*>(s.data()),
            s.size_bytes()
        );
    }
} // namespace std2

namespace std::ranges {
    template<class ElementType, size_t Extent>
    constexpr bool enable_view<
        ::std2::span<ElementType, Extent>
    > = true;
    template<class ElementType, size_t Extent>
    constexpr bool enable_borrowed_range<
        ::std2::span<ElementType, Extent>
    > = true;
}

bool bool_true() { return true; }
bool bool_false() { return false; }

```

```

int main() {
#ifdef PROPOSED_CHANGE
{
    std2::span<const bool> b{true, false, true};
    assert(b.size() == 3);
    assert(b[0] && ! b[1] && b[2]);
}
{
    std2::span<const bool> b{bool_true(), bool_false(), bool_true()};
    assert(b.size() == 3);
    assert(b[0] && ! b[1] && b[2]);
}

//{
//    std2::span<const bool> b{1, 0, 1};
//    assert(b.size() == 3);
//    assert(b[0] && ! b[1] && b[2]);
//}
//{
//    std2::span<const std::any> b{"foo", true, 1.25f};
//    assert(b.size() == 3);
//}
{
    std2::span<const std::any> b{
        std::any{"foo"}, std::any{true}, std::any{1.25f}
    };
    assert(b.size() == 3);
}
{
    std2::span<const float> b{1.0f, 2.0f, 3.0f};
    assert(b.size() == 3);
    assert(b[0] == 1.0f && b[1] == 2.0f && b[2] == 3.0f);
}
//{
//    std2::span<const float> b{1.0, 2.0, 3.0};
//    assert(b.size() == 3);
//    assert(b[0] == 1.0f && b[1] == 2.0f && b[2] == 3.0f);
//}
//{
//    std2::span<const float> b{1, 2, 3};
//    assert(b.size() == 3);
//    assert(b[0] == 1.0f && b[1] == 2.0f && b[2] == 3.0f);
//}

#endif

    return 0;
}

```